Anton Gerklev
Omniway id: 99695

Datastructures and Algorithms

Weekly progress

Week 1

The start of the course felt quite easy, making a custom queue was not really a challenge, so instead I worked on making a custom list and then using that for my queue. This took a little bit longer to figure out but was a great exercise. With this in place making a stack was pretty straightforward, and added some extra just funny functions to it because I was bored.

Week 2

After a bit of contemplating I came up with a small game concept which would propel my enjoyment for the course. I passed off this concept to the teacher to get it approved. Reading from a Json was something I was well familiar with, well not the textasset way of reading which made the whole process a lot easier.

Week 3

In this week things turned from quite easy to harder to grasp, keeping track of what happens in the more difficult sorting methods such as Quicksort and Merge sort baffled my mind and I struggled on implementing these. Came down to me searching for different youtube videos to really watch the process step by step to get a better understanding how these were performed. I also spent a great deal of time outside of Unity making short projects where I could test out my implementations. After I got my methods working with numbers I started looking at ways to make this sorting with any types aka generic.

This is where large experiments started, I figured out a way to use reflection to figure out what types of variables were stored in a class and then contained that information in a special class and just parsed these variables of the array and then matched these by unboxing and boxing variables. I learned that this was a very costly performancewise to do during sorting and eventually threw this concept to the side.

So I started looking into how the ICompareable and IComparer, but it was difficult to get a good answer about this since I could not clearly see the code. Therefore I started applying the generic logic we had applied before and saw that you could add many generics to the functions which led to creating a IClassCompare that would take in the Type of class and an Enumerator that would point to the correct field when called upon. This worked great and opened up a whole new world of syntaxing.

Week 4

After last week's discovery I could now easily search on all the fields in my class by passing an enum that was tied to the class. So for this week I mainly started working on the tree structure, but I had a very hard time understanding what the use for this would be since I already figured out a way to store my sorted lists in an array that formed in the positions my original list would look like sorted and since we were never supposed to remove employees from the collection the main use of a tree seemed somewhat unnecessary.

My first attempt at making a tree structure was very counterintuitive, my idea was to store the sorted indexes in the tree so that would place them in a sorted manner, after making this I could not really figure out how to use any of this data in any meaningful way since returning values at every node visit would break the speed of searching in a tree or just make navigating in the tree extremely difficult. So instead I went with storing my employees in the different nodes and then passing the IClassCompare to be linked to the type the tree was created for, then I made a tree structure for every field.

The tree structure was the hardest part of the course, but mainly because I could not find a good reason to use it and the way to navigate in the tree was complex to hold in my head while troubleshooting.

Week 5

The final week was spent on tying up loose ends since I had all the pieces already in place, but I thought it would be fun to make a measurement box for the teacher when going through the projects. So I created a little Ui system to reflect time measurements of the different functions. I had large ambitions for this where I wanted to make measurements for sorting methods of the different datatypes but eventually scrapped this part since I had to spend time writing this document.

I also noticed the updated assignment document saying we had to find the actual suspect, So below are the searches for last entrance and last exit. Since there is only one suspect in both results the 'killer' is Alfred Pugsley *dramatic chipmunk*

514. Beatrice Graves - Last entrance: 2234-08-03T12:51:23 > Last exit: 2234-08-09T22:14:54Binary range:
118. Arthur Holloway - Last entrance: 2234-08-02T06:11:28 > Last exit: 2234-08-09T22:14:54Binary range:
192. Alfred Pugsley - Last entrance: 2234-08-09T22:13:17 > Last exit: 2234-08-09T22:14:54Binary range:
792. Alfred Underwood - Last entrance: 2234-08-08T21:03:38 > Last exit: 2234-08-09T22:14:54

47. Jasper Wellington - Last entrance: 2234-08-09T22:13:17 > Last exit: 2234-08-04T07:14:06Binary range:
192. Alfred Pugsley - Last entrance: 2234-08-09T22:13:17 > Last exit: 2234-08-09T22:14:54Binary range:
764. Helena Blackwood - Last entrance: 2234-08-09T22:13:17 > Last exit: 2234-08-03T09:43:27Binary range:
288. Nem Wellington - Last entrance: 2234-08-09T22:13:17 > Last exit: 2234-08-02T09:16:49

Detailed implementation

Range searching

My first idea to implement this function was to just run a binarysearch twice once for the low and one for the high. Put the indexes into an array with the bounds. This did not work out quite as I expected since it did not grab the duplicated values of the date 1970.
So instead I worked on a binarysearch that would keep going until it actually find all most lowbound and highbound value, then it would return the indexes these were placed at.

This worked almost the same as the binary search I pass the array make a comparison at the middle point against the lowSearchValue and set lowpoint to middle +1 if the comparison value was less than one. Otherwise I knew that the values were to the right instead so I would make the highpoint into middle. And then get the middle of lowpoint to highpoint at every iteration. When the loop had reached its exit it would return the lowest index position for both the higherbound search and lowerbound search.
Flip these if the lowindex was higher than highindex. Return a tuple of the values and feed this into a new array of employees. (SearchRangeTask.cs)

Advanced Datastructure: Tree

The final tree structure would be defined as BinaryTree with a generic T which would be the class and an E which would be the Enumerator tied to IClassCompare attached to the class. The nodes will be defined with the T as it should carry the data of Tenant. With this in place my tree knew that the T type will have to carry the IClassCompare so comparing data would be possible.

Then when setting up the tree I prepare the different structures of data in (StructurePrepareClass.cs) Sorts the class on all its fields and add them into dictionaries for quick access, I could have used arrays here and stored them based on the enum value, but thought this seemed cleaner.

Then take a sorted array and start the take root on the BinaryTree inserting the midpoint and recursively adding middle -1 and middle +1. This is to keep the tree balanced on the side to ensure that searching speed stays optimized. The adding of the different nodes is handled by the Branch function which just compares the current node data to the incoming one and heads left or right based on the compare value. Making sure to shove duplicates to the right side.

Then we move into the DFS implementation we use a stack to be able to backtrack. Have currentnode that starts at the root. Setup a while loop that will run as long as currentnode is not null or the stack.count is above 0. After that we check again if the currentnode is not null, then we add that node into the stack (this makes sure that we don't pop anything that is null). Then we check if the data on the current node against the low range, if the node value is above the lowrange we keep going left, by doing currentnode = currentnode.left. If the value is lower we cut off this part of the tree and stop heading left by breaking out of the loop.

Now we pop the stack to get the last added node, check if this data is within the range of both low and high, if it is we add it to the result pool. After this we make check to see if the compare high value is less than 0 or equal, this is to grab duplicates pushed to right side if equal otherwise we set the currentnode to null, and continue popping the stack. This makes sure we visit the right side as well and add all objects to the left and check these first. If the right node is null, we just pass through the left checking and pop the stack.

Finally when we reached a node that is above the high range value we set the currentnode to null to continue popping nodes.

When searching in a BFS manner we deploy a queue so we visit all the nodes in a level to level basis. So how I worked this out is by queuing the root into the queue and running a while loop while the queue is not empty. We dequeue the value into the current node and then check if currentnode is not null, Then we do a comparison on both low and high to the data, if the value is within the two values we add into the result pool. If the compare low value is greater than 0, we queue the left node and if the compare high value is lower or equal to 0, queue the right node.

Things to note between BFS and DFS is that getting the results in a DFS method returns all the values in a sorted order while BFS steps from level to level in the tree and inserts them in that order which causes the return values to be in a shuffled manner.

Generic Searching

As previously mentioned I used a special way of getting this working first by getting the fields of a class and repacking that information into a class, where the variable would be collected as an 'object' this causes the compare functions needing to unbox the object into the correct type at every iteration which can be quite slow.

So to get around this issue I made a IClassCompare interface that would be attached to the class that I wanted to be able to compare against. In the interface I defined a T and E where T is the class and E is an Enum that would in a switch point to the correct variable.

This interface includes 3 abstract functions

CompareThis which takes in a T as b and E the enum. So when this function runs I make a switch case that points to different variables and then just perform a simple compare for the specific variable and returns an int of -1,0,1.

The second CompareData<D> takes in D as the data that is being compared against and the enum again, this works in a similar manner of comparing the values against the correct part of the class. The downside of this is that wrong datatypes to the incorrect enum can cause issues. Therefore when sending in data I had to make sure it was the correct type.

The third one was PrintClass which returned a string, this is just to easily debug when the class was stored inside a binary tree.

Code for this part is here.

With this interface I was able to sort my data by including the T and E in a function and adding at the end Where T: IClassCompare<T,E> which limits the function to only use data types that include that Interface.

Measurements

Many of my measurements were done outside of this unity project with larger datasets on the sorting methods to test out some differences when using multithreading and differences between ints and strings. Where a string comparison is a much larger and heavier task to perform.

But when talking about the datasets of the provided JSON specifically the searching of a range of objects. The BFS is a little and I mean a little faster than the DFS search, This speed is lost when taking into account that the values returned are not returned in an ordered manner. But when using this structure for something that just needs to update a range of objects the order does not really matter that much, so there is definitely use cases for this method.

The reason this might be is that the BFS pattern is using a little less comparisons when figuring out when to go and cutting off larger parts of the levels when traversing.
Still the binary range search function blows these out of the water in comparison since it is using an array which have their memory locations stored in order. Causing it to be lightning fast, the true power of the tree structure is shown when comparing speeds of adding new elements instead of an array where resizing is required. This type of comparison is something I did in a separate project when testing much larger datasets.

The measurements for finding objects in the different structures are inside the project with some graphical elements to make a little more pretty to look at.

When doing the measurements I made sure I took them in a similar fashion the code for this is here. The way I did it was creating a function that returns a Stopwatch and takes in a action. Then when calling this function I collected the stopwatch that started just before the action was fired and stopped right after. This stopwatch was then collected into a struct that just kept a name and the elapsed ms and elapsed ticks. After all measurements were done I sent a list of these structs and constructed my measurement sticks displaying the values.

Extras

When doing this course I started doing a couple of extra stuff worth mentioning.

- A custom list - Was going to use this for everything but when extracting information from a json only supported datatypes were allowed and my custom list was not. A teacher mentioned I had to write my own json parser for this. Which felt out of scope for this.
- A datetime converter - This was to alleviate searching on higher ranges, since when inputting just a year and month it would default to the first day of the month in both the low range (which works great) but the same would happen for the high range. This would result in the search not fetching everything in that month. In this I also added the functionality of just inserting the year as that would not parse correctly into a datetime.
- Generic Class Sorter - The conversion of fields into a package and then searching based on an integer to find the correct field. Mostly an exploration into how reflection works. This project might be in quite a disarray since it's been my goto (let's try something random project)

Reflection

Mostly this course I felt that my progress was on par, I learned a bunch of new things but the actual game project suffered as a result of trying to understand how the tree structure worked. Traversal in the tree was very complicated at the start and it took a few iterations until I finally got it working correctly.

Many of the struggles I encountered were of a comparing values nature, where I struggled to keep what value was compared to what, this took quite a lot of trial and error to figure out what was going on so this is certainly something I need to improve upon.

Even though I'm happy with the end result of the actual assignment, I'm a little sad that I didn't have time to flesh out the actual game.